COS30018 - Option C - Task 5 Report

Machine Learning 1

Student Name: Kha Anh Nguyen
Student ID: 103814796
Date: September 5, 2025
Course: COS30018 - Intelligent Systems
Project: FinTech101 Stock Price Prediction System
Github repo: Leissha/stock_prediction

Table of Contents

The prediction days parameter and multiple input features were already set up in previous labs. The detailed code explanation can be found in Task 2's data preprocessing section. This report documents the code updates needed to support multivariate and multistep predictions, along with the evaluation results.

1. Key changes:

- I updated the **create_sequences** function, **DataProcessor**, **TFModel** class and **train** function to accept the "lookup_steps" parameter through the entire pipeline for multistep prediction support.
  (I set the lookup_steps to be interger = 1 as default before).
- In addition, I refactor & separate the main training code file into 3 files, each with distinct functionalities:
  1. main.py: Handles command-line arguments and coordinates the entire process
  2. train.py: Contains the model training logic
  3. test.py: Handles model prediction and evaluation

2. Multivariate Prediction

Recap Week 2 report – adapt from P1 code

The input features accept 5 values, tickers at: Close, High, Low, Open and ticker's Volume

```
# Input features (OHLCV): ['close', 'high', 'low', 'open', 'volume']
training_features = ['close', 'high', 'low', 'open', 'volume']
```

I also use separated scaler for each feature at data preprocessing:

```
            for i, feature in enumerate(training_features):
                # Create and store individual scaler for each feature
                scaler_key = f"{ticker}_{feature}"
                self.scalers[scaler_key] = StandardScaler()

                # Fit scaler only on training data to prevent data leakage
                train_scaled[:, i] = self.scalers[scaler_key].fit_transform(
                    train_df[[feature]].values
                ).reshape(-1)
                test_scaled[:, i] = self.scalers[scaler_key].transform(
                    test_df[[feature]].values
                ).reshape(-1)
            # Cache all feature scalers for future inference use
            scaler_bundle = {
                "scalers": self.scalers,     # per-feature scalers dict
                "training_features": training_features,      # column order
                "target_scaler_key": f"{ticker}_{target_feature}",  # just the
key string
            }
```

3. Multistep predictions

Create sequence

I add a functionality block to turn the integer prediction day to an array of prediction days integer.

```
    # Window bounds:
    # start at i = lag_days so there is enough history for the first window
    # stop at len(scaled_data) - lookup_steps to allow indexing i + step + 1
(t+1..t+k)
    for i in range(lag_days, len(scaled_data) - lookup_steps):
        # Historical sequence: all features for past lag_days
```

```
        X.append(scaled_data[i-lag_days:i, :])

        # Future target: create sequence of future values (t+1 .. t+k)
        future_sequence = []
        for step in range(lookup_steps):
            future_sequence.append(scaled_data[i + step + 1, target_indices])

        # Always return as array for consistency
        y.append(future_sequence)

    X = np.array(X)  # Shape: (n_samples, lag_days, n_features)
    y = np.array(y)  # Shape: (n_samples, n_targets) or (n_samples, lookup_steps,
n_targets)
```

TFModel Base Model:

The base model also store the prediction days length attribute...

```
def __init__(self, input_size: int, model_name: str = MODEL_NAME, layers:
Optional[List[int]] = LAYERS,
             dropout_rate: float = DROPOUT, output_steps: int = 1):
    # ...
    self.output_steps = int(output_steps)
```

So the build model output layer can be flexible:

```
def _build_model(self):
    # ...
    # Final prediction layer
    # When output_steps=1: units=1 (single prediction)
    # When output_steps>1: units=output_steps (multiple predictions)
    model.add(Dense(units=self.output_steps, activation='linear'))
```

Test:

Test function simply need to add additional condition to identify & extract last step for multistep 3D array

```
def test(model, x_test, y_test, data, ticker, use_log_returns: bool,
target_as_return: bool):
    """
    Produce actual/predicted price series for the full test set, handling
```

```
    inverse-scaling and return-to-price reconstruction in one place.
    Returns (actual_prices, predicted_prices).
    """
    # Get target feature name and corresponding scaler for inverse transformation
    tf_key = data['target_feature'].lower()
    target_scaler = data['scalers'].get(f"{ticker}_{tf_key}")

    # Get predictions once (works for both single-step and multistep)
    predictions = model.predict_and_evaluate(x_test, y_test)[0]

    # Extract the last step for both single-step and multistep
    # For single-step (2D arr): predictions shape (N, 1) -> predictions[:, -1] =
(N,)
    # For multistep (3D arr): predictions shape (N, k) -> predictions[:, -1] =
(N,) (last day (kth) prediction)
    predictions = predictions[:, -1]
    y_test = y_test[:, -1, 0] if len(y_test.shape) == 3 else y_test[:, -1]
```

Key Logic Explanations:

1. **Valid Sequence Calculation**:

This ensures we only create sequences where we have enough future data to validate our predictions.

```
    # Calculate number of valid sequences (accounting for horizon)
    # E.g. If we have 100 days of test data and want to predict 5 days ahead:
    #       We can only create sequences that start at day 1 and end at day 95
    #       Because sequence starting at day 96 would need days 97, 98, 99, 100,
101 (but day 101 doesn't exist)
    num_valid_sequences = max(0, len(y_true_df) - lookup_steps) # ensure no
negative values
```

2. **True Price Sequence Construction**:

This builds a matrix where each column represents prices at a specific

horizon (t+1, t+2, ..., t+k).

```
    # Get starting prices y[t] at time t to create future sequences
    y_t = y_true_df[:num_valid_sequences]

    # Build true price sequence: y[t+1], y[t+2], ..., y[t+k]
    # Each column h represents prices at horizon h (t+h)
```

```
    #   Original data: [100, 102, 105, 103, 101]
    #   Prediction horizon: 3 days

    #   Sequences we can create:
    #   Sequence 0: base=100, future=[102, 105, 103]
    #   Sequence 1: base=102, future=[105, 103, 101]

    #   y_seq matrix:
    #   [[102, 105, 103],    # Sequence 0: prices at t+1, t+2, t+3
    #    [105, 103, 101]]    # Sequence 1: prices at t+1, t+2, t+3

    y_seq = np.column_stack([
        y_true_df[horizon:horizon + num_valid_sequences]
        for horizon in range(1, lookup_steps + 1)
    ])  # create matrix of shape: (N_sequences, k_lookup_steps)
```

3. **Return Compounding Logic**:

Since the % & log return are based on previous day changes, we need cumulation for returns

- **Log returns**: y[t+k] = y[t] × exp(∑r[t+i]) where the sum is over i=1 to k
- **Simple returns**: y[t+k] = y[t] × ∏(1+r[t+i]) where the product is over i=1 to k
    - ⇨ Uses np.cumsum() for log returns and np.cumprod() for simple returns

```
    if target_as_return:
        # CASE 1: Model predicts returns, need to compound to get prices
        y_hat_returns = raw_scaled_predictions # Predicted returns sequence(N, k)

        # Same old inverse-transform logic for % & log ret…

        # Compound returns to get price paths
        if use_log_returns:
         # For log returns: y[t+k] = y[t] *exp(sum(r[t+1] + r[t+2] +...+ r[t+k]))
            cumulative_log_return_factors = np.exp(np.cumsum(y_hat_returns,
axis=1))
        else:
         # For % returns: y[t+k] = y[t] * (1+r[t+1]) * (1+r[t+2]) *...*(1+r[t+k])
            cumulative_return_factors = np.cumprod(1.0 + y_hat_returns, axis=1)
            cumulative_log_return_factors = cumulative_return_factors

        # Apply compounding: multiply base prices by cumulative factors
        Y_hat = y_t[:, None] * cumulative_log_return_factors
```

```
    else:
        # CASE 2: Model predicts prices directly
        Y_hat = raw_scaled_predictions

        # Same old inverse-transform logic for price ret…

        # Ensure we only take valid sequences
        Y_hat = Y_hat[:num_valid_sequences, :]
```

5. **Final Horizon Extraction**:

Returns both the full sequences (for per-horizon analysis) and the final horizon prices (for overall evaluation).

```
    # Extract final horizon prices for evaluation (t+k)
    y_k = y_seq[:, -1]  # True prices at t+k
    y_hat_k = Y_hat[:, -1]  # Predicted prices at t+k

    return (y_k, y_hat_k, y_seq, Y_hat)
```

This implementation correctly handles the multistep prediction challenge by:

- Building proper future-only target sequences
- Compounding returns across the prediction horizon
- Providing both full sequences and final prices for comprehensive evaluation

The key insight is that for return targets, we must compound the predicted returns to reconstruct the price paths, while for price targets, we can directly inverse-transform the predicted prices.

4. Results

CLI

Multivariate inputs are already integrated in the pipeline, we can use --lookup_steps arg to make different prediction days:

```
### Combined Multivariate-Multistep
python main.py --company CBA.AX --lookup_steps 3 --epochs 200 --target_ret // 5 days return
```

or powerShell batch runs:

```
# Usage: Right-click → Run with PowerShell
# or cd/dev & run: powershell -ExecutionPolicy Bypass -File run_experiments.ps1

param(
  [string]$Company = "CBA.AX",
  [string]$StartDate = "2023-10-01",
  [string]$EndDate = "2025-09-30",
  [int]$LagDays = 60,
  [int]$Epochs = 200,
  [int]$BatchSize = 32,
  [string]$ModelName = "lstm",
  [int[]]$ReturnSteps = @(1,5,10),
  [int[]]$PriceSteps = @(1,5)
)
```

Test Result Plots

Noted that I use direct multi-horizon prediction, not recursive to reduce the error accumulation. The results have 2 main plots:

- **Training plots:** Loss (MSE) and MAE over epochs to check convergence/overfit.
- **Prediction plots:** Actual test (black) vs Predicted (green), time-aligned with test_df.index[-len(actual):] and inverse-scaled using the target scaler.s

LSTM model with lookup-steps = 1, 5 and price return:

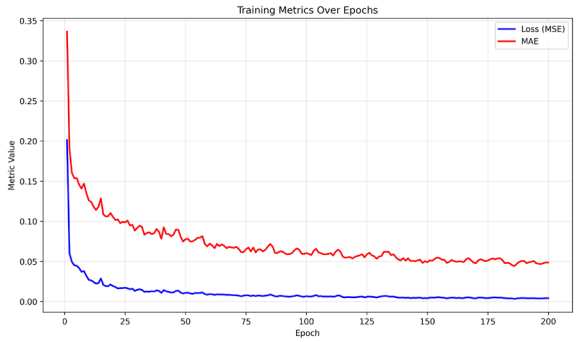

Figure 1. Training plot for LSTM epochs 200, multivariate features, single step price prediction

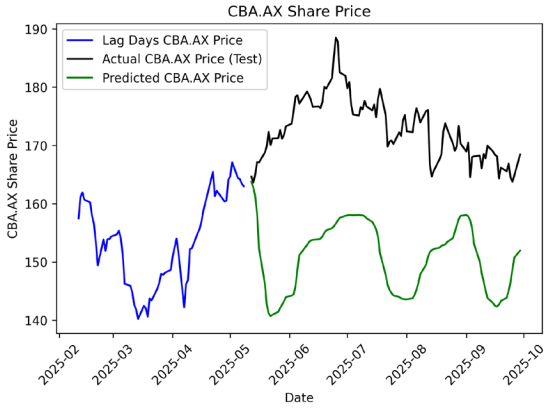

Figure 3 Prediction plot for LSTM epochs 200, multivariate features, single step price prediction

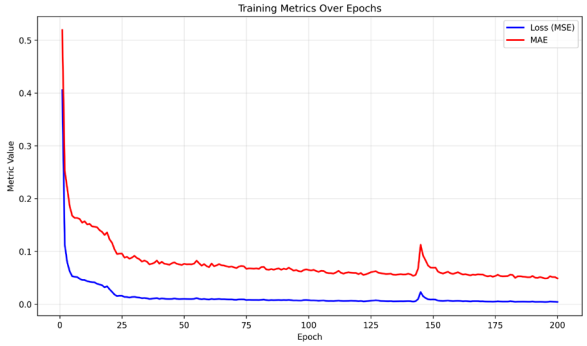

Figure 2 Training plot for LSTM epochs 200, multivariate features, with price prediction and lookup steps = 5

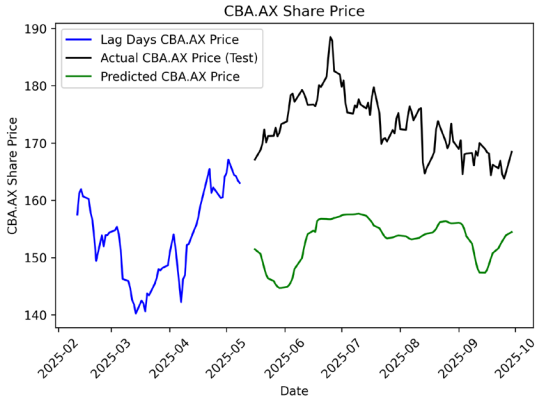

Figure 4 Prediction plot for LSTM epochs 200, multivariate features, with price prediction and lookup step = 5

LSTM model percentage return predictions with layers = [50,50,50], epochs = 200, and lookup-steps = 1, 5, 10

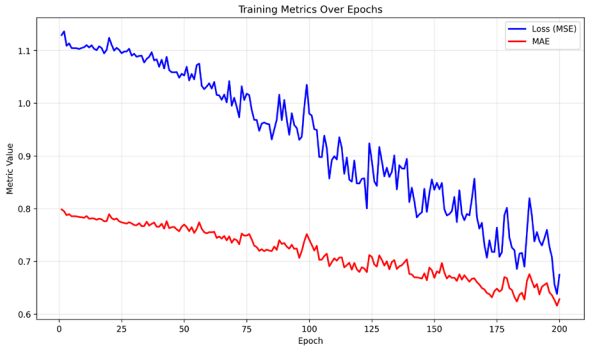

Figure 5 Lookup_steps = 1 with percentage-change target return

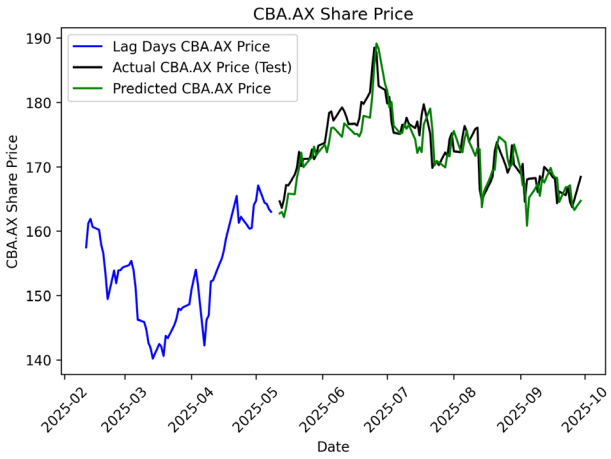

Figure 6 Lookup_steps = 1 with percentage-change target return

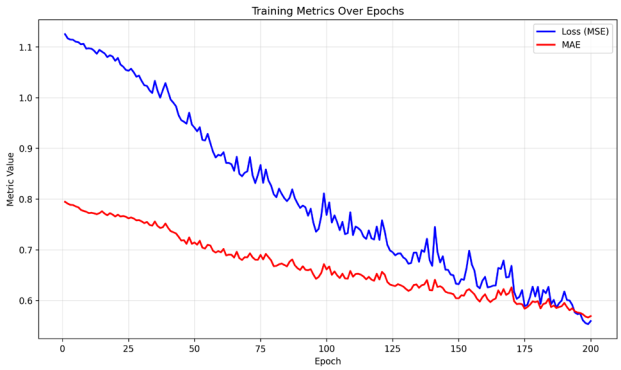

Figure 7 Lookup_steps = 5 with percentage-change target return

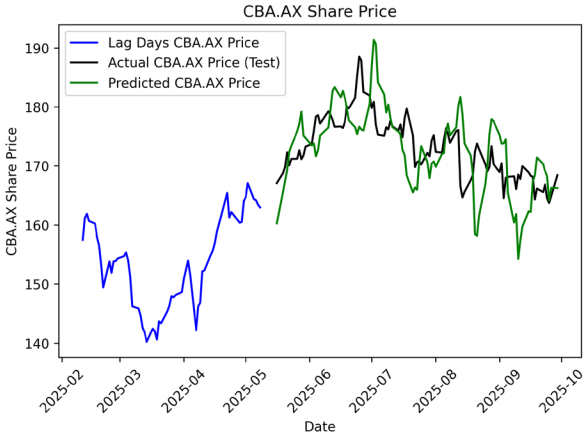

Figure 8 Lookup_steps = 5 with percentage-change target return

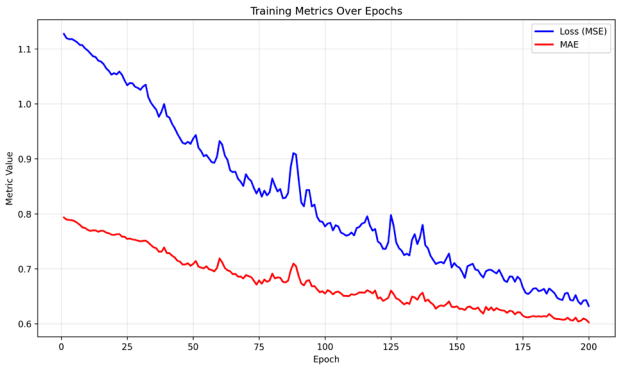

Figure 9 Lookup_steps = 10 with percentage-change target return

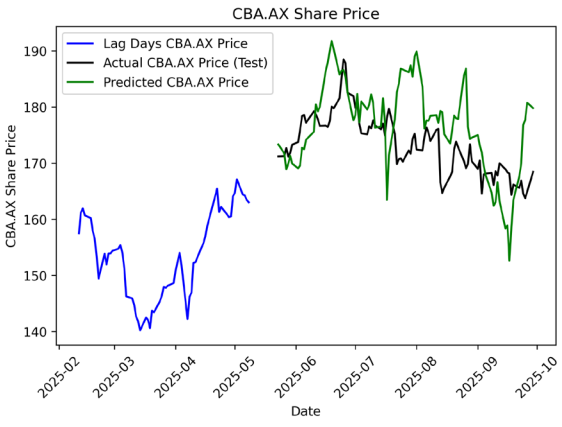

Figure 10 Lookup_steps =10 with percentage-change target return

Statistic summary and justification

Convergence (training curves)

All runs show **smooth loss/MAE decay** over 200 epochs with no late-epoch divergence. Implies the architecture, learning rate, and regularization are appropriate for both single-step and multi-step settings.

Forecast plot shape

- **k = 1 (price)**: Significant systematic underestimation and smooth out predictions, however follow the true trend.
- **k = 1 (return)**: Well-correlated with actual price movements but still can't captivate the peak changing well. Has good directional accuracy but misses sharp peaks and rapid market changes
- **k = 5 (price)**: Even greater systematic underestimation with more error in trend predictions. Both price level and trend accuracy degrade compared to k=1.
- **k = 5 (return)**: Trend predictions lagged but similar to actual price trends, can't captivate small fluctuations.
- **k = 10 (return)**: 10-day lookup horizon (with 60 lag days) is too long for the model to maintain any meaningful connection to actual price dynamics. The model failed to adapt to current market movements, instead its prediction shows similarity in historical lag days patterns, which signals overfitting.

Table features:

- **MAE/RMSE** are computed on true price k days later **final horizon** ($t + k$).
    - If we predict **price**, the errors are in **dollars**.
    - If we predict **return**, the errors are in **%/return units.**
        - ⇨ Hence, MAE is different across price vs return.

Model	Steps	Target	MAE	RMSE
LSTM	1	Price	22.043	23.115
LSTM	5	Price	20.018	20.753
LSTM	1	Return	2.069	2.604
LSTM	5	Return	4.996	6.325
LSTM	10	Return	6.410	8.173

5. Conclusion

The corrected multivariate multistep pipeline successfully implements future-only labels and proper return compounding, yielding consistent k-step evaluation across different prediction horizons.

Return targets consistently outperform price targets across all horizons due to better scaling and relative change focus, while price targets suffer from systematic underestimation and level bias issues.

With only basic stock data (Open, High, Low, Close, Volume) and no additional market indicators, the model works best for short-term predictions of 1-2 days ahead. These short horizons provide real forecasting value while avoiding the problem where longer ssspredictions lose connection to current market conditions.

The model trains consistently across all prediction lengths, but has a practical limit around 5 days. Beyond this point, the model starts predicting based on old historical patterns instead of current market movements, which is why longer predictions become unreliable.

This highlights a key challenge in stock prediction: as we try to predict further into the future, the model's ability to stay connected to current market dynamics decreases, making longer-term forecasts less accurate